PA2: AWS and Dask Setup

Released: 12 Oct 2026, Due: 30 Oct 2026

Contents

1 Introduction 1

2 AWS Setup 1

3 Dask Setup 4
- 3.1 First Instance Setup . . . 5
- 3.2 Remaining 4 Instances Setup . . . 5
- 3.3 Sanity Check . . . 6

4 Large Language Model Usage 8

1 Introduction

This manual will cover how to setup multiple instances on AWS and start the Dask scheduler and workers for Programming Assignment 2.

For this assignment, you will be running your code on two EC2 instances and also on a cluster of five EC2 instances and measure the speedup. There are points for how much speedup your code achieves when moving from 2 to 5 instances. Refer to the grading rubric for more details.

2 AWS Setup

The AWS setup is very similar to PA1. For launching multiple instances, there are three differences:

1. We will specify 5 instances instead of 1. One of these instances will run our "Jupyter Notebook" and "Dask Scheduler", and the remaining 4 instances will run our "Dask workers".
2. Each of these instances will have 100GB SSD storage instead of 40GB.
3. We will create a new security group for our 5 instances that allows each of the instances to communicate with each other.

Follow the below steps one by one.

a. Access your ETS account using single sign-on ID: `https://ets-apps.ucsd.edu/individual/DSC102_FA26_001`. To open the AWS console click "Click here to access AWS" at the bottom of the page. To get your AWS credentials for CLI/API usage click "Generate API Keys (for CLI/scripting)".

b. Open AWS Dashboard. We will first create a new security group so that we can apply it to all of our instances later. Click on "Security Groups" on the left menu.

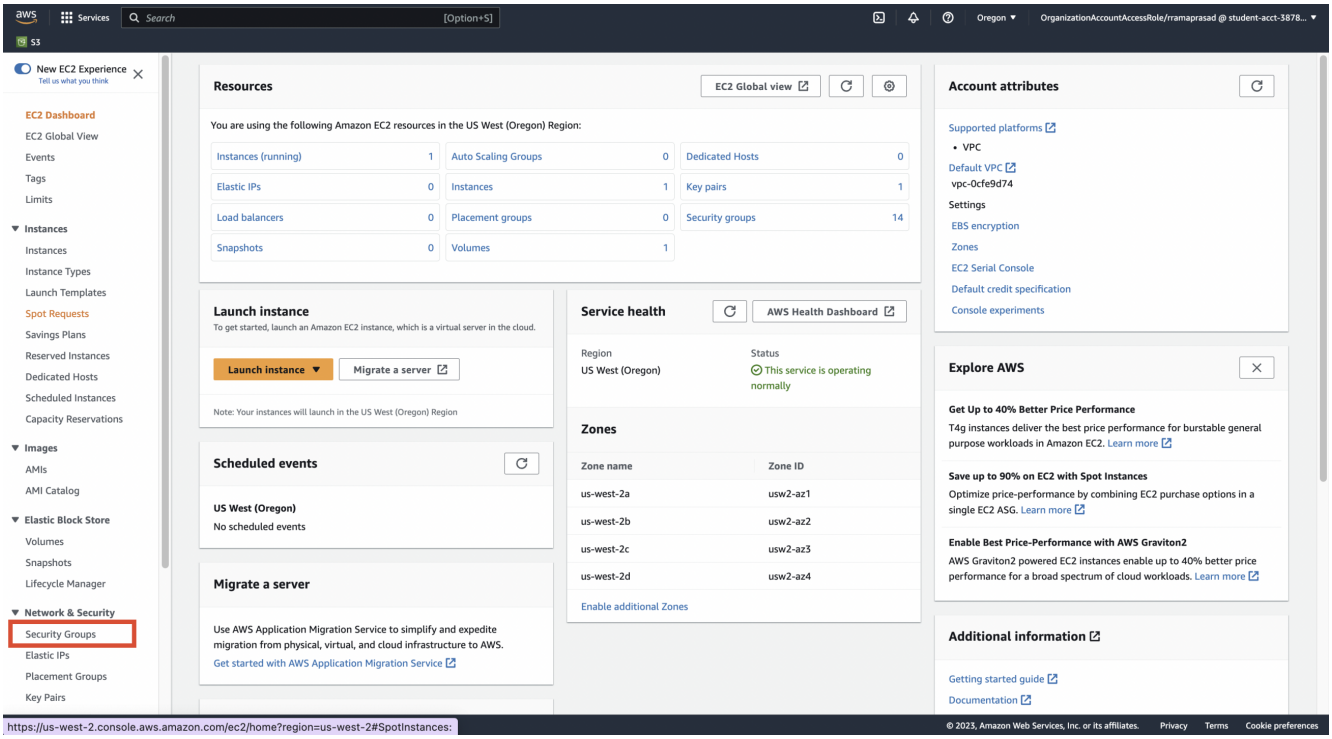


Figure 1: AWS Dashboard – Security Groups

c. Click on "create security group".

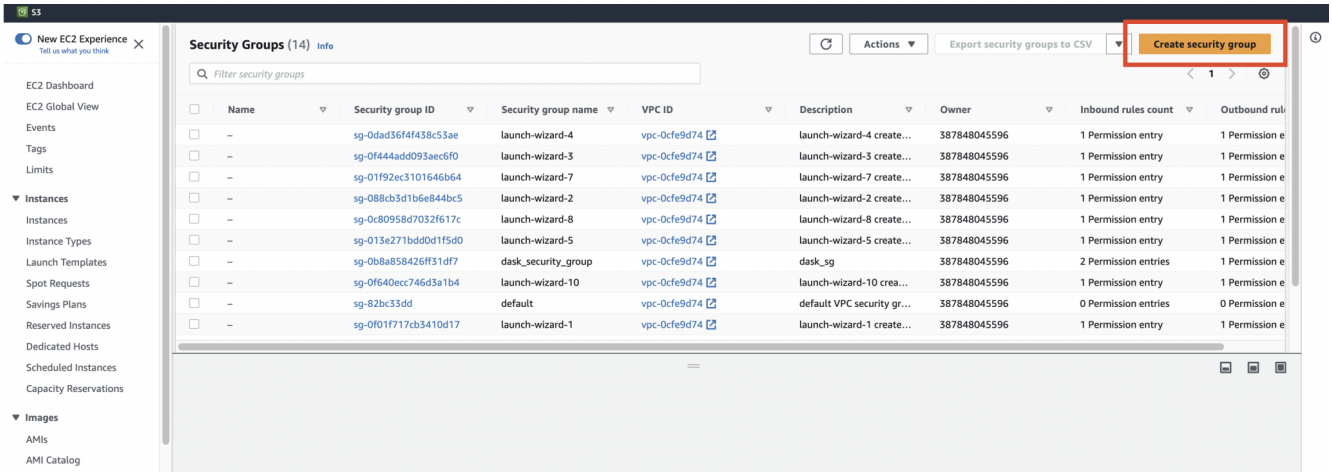


Figure 2: Create Security Group

d. Choose any name and description for your security group. Change both the inbound and outbound rules to have "Type=All Traffic", "Source/Destination = Anywhere-IPv4". See image below.

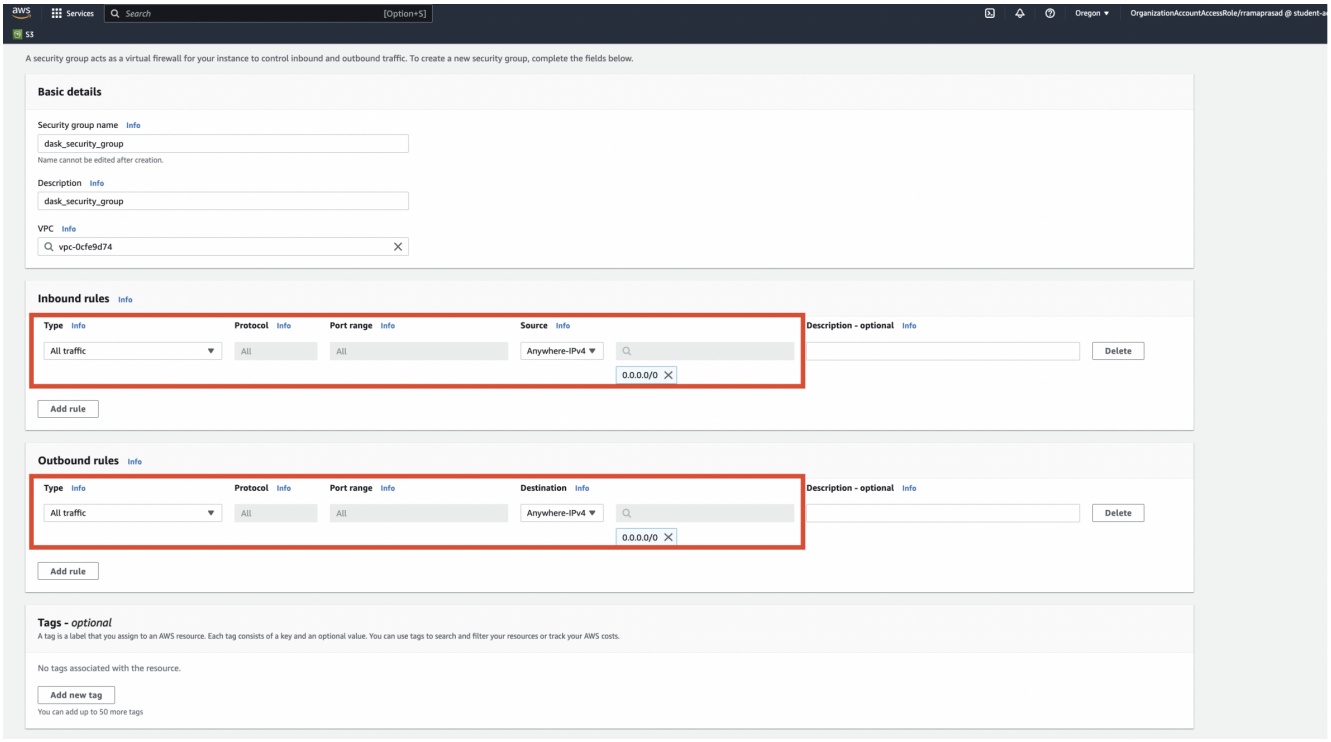


Figure 3: Set Inbound/Outbound Rules

e. Lastly, click on "Create Security Group" at the bottom right.

f. Now, we will create our 5 instances which will use this new security group. We have setup the Dask environment on an AMI with the name "dsc102-dask-environment-public". Go to "AMIs" (under "Images") in your EC2 dashboard, select private images, and then search by name to find it. Select this AMI. See Figure 4.

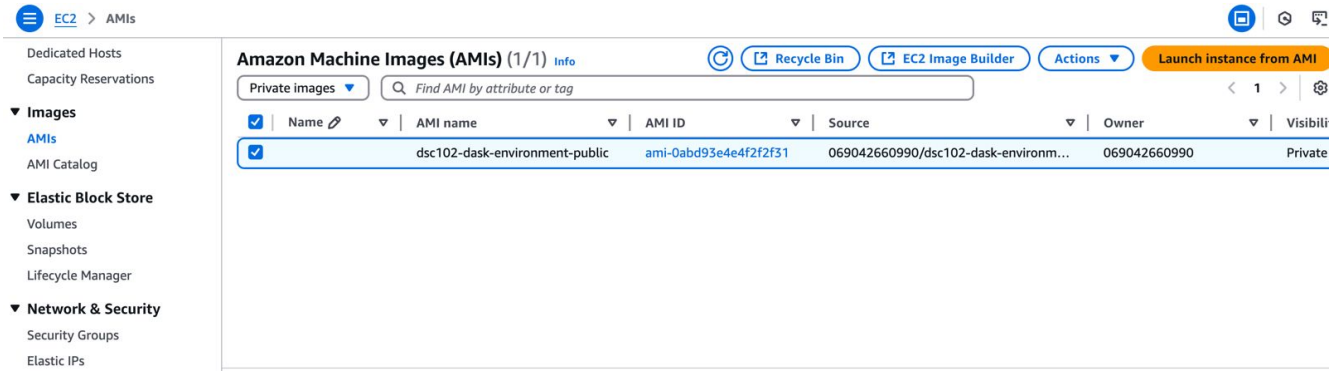


Figure 4: Select AMI

g. After selecting the AMI, click "Launch Instance from AMI" as shown below.

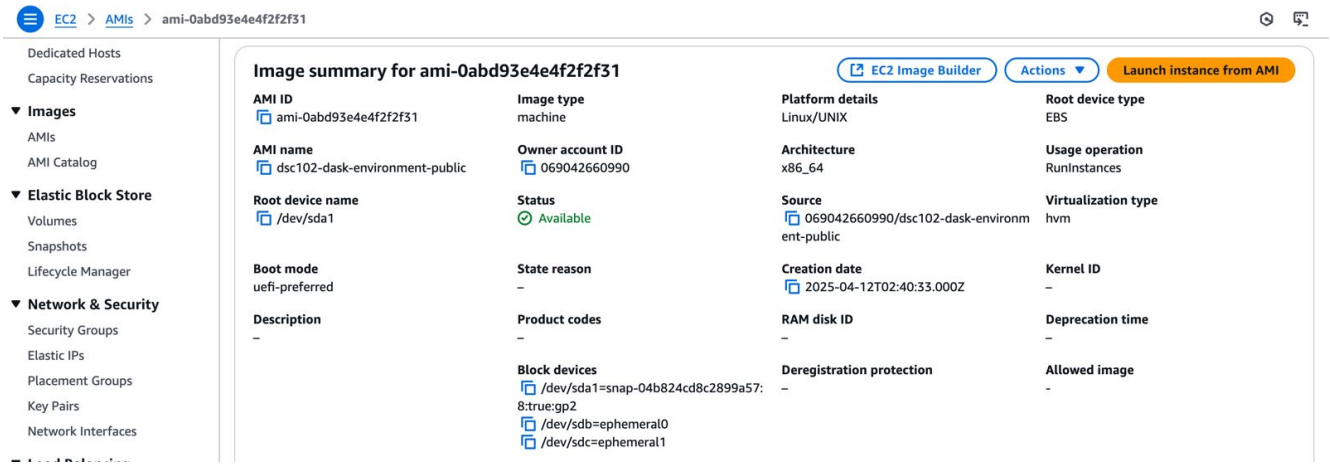


Figure 5: Launch Instance from AMI

h. Now, strictly follow the below instructions to launch the EC2 Spot instances.

   a. Give any name for your instance.

   b. Number of instances to launch is 5.

   c. The instance type is "t2.xlarge".

   d. Create a new key pair for SSH'ing to your instance later. The private key will be downloaded to your local machine.

   e. **Under "Network Settings", click "Select Existing Security Group" and choose the name of the security group you just created, e.g., "dask_security_group".**

   f. Choose 100GB SSD gp2 storage.

   g. Open advanced details. Select "Request Spot Instances".

   h. Lastly, click "Launch Instance".

3 Dask Setup

After launching 5 instances, your AWS dashboard must look like the figure below. You can view the IP addresses of the different instances inside the red box.

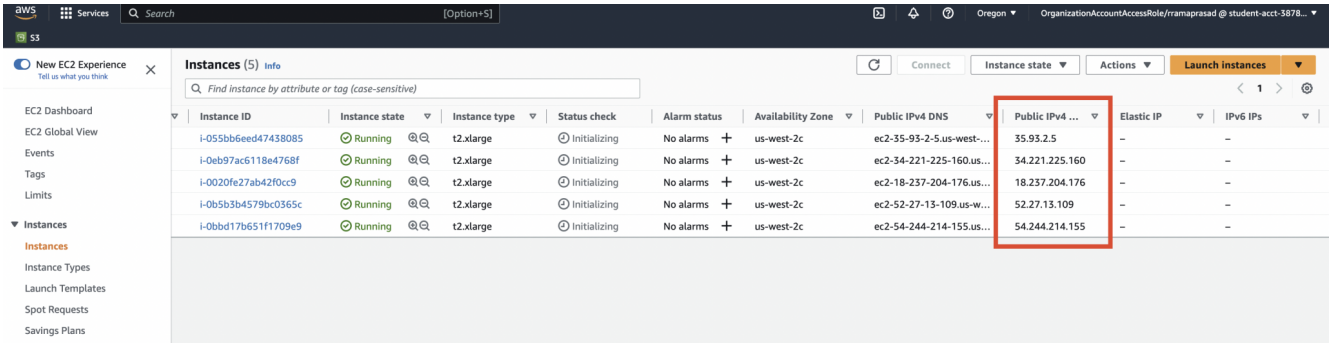


Figure 6: AWS Instances Overview

3.1 First Instance Setup

We will run "Jupyter Notebook" and the "Dask scheduler" on the first of our 5 EC2 instances. As you will see, we will SSH into this instance 5 times on 5 different terminals to execute different commands. If you are comfortable with tmux you can avoid creating these many terminals, but for sake of simplicity, we will not use tmux. The IP address of the first instance is, for example, 35.95.2.5. **Replace this IP with the IP of your first instance in the commands below**.

1. **Terminal 1 - Dataset transfer from S3 bucket into our EC2 SSD**
   (a) Open a terminal and SSH into the instance:
   `ssh -i "my-key.pem" ubuntu@35.95.2.5`
   (b) Copy your AWS API Keys (from the link on ETS dashboard) and paste them in the terminal.
   (c) Start dataset transfer:
   `aws s3 sync s3://dsc102-pa1-public .`
2. **Terminal 2 - Starting jupyter notebook**
   (a) SSH into the instance.
   (b) Activate the dask environment:
   `source dask_env/bin/activate`
   (c) Start jupyter notebook on port 8888:
   `jupyter-notebook --port 8888`
3. **Terminal 3 - Starting Dask Scheduler**
   (a) SSH into the instance.
   (b) Activate the dask environment:
   `source dask_env/bin/activate`
   (c) Start Dask scheduler:
   `dask scheduler --host 0.0.0.0`
4. **Terminal 4 - Port forward jupyter notebook**
   (a) Port forward EC2's port 8888 (jupyter notebook) to your computer's port 8888:
   `ssh -i "my-key.pem" ubuntu@35.95.2.5 -L 8888:localhost:8888`
5. **Terminal 5 - Port forward dask dashboard**
   5.1. Port forward EC2's port 8787 (dask dashboard) to your computer's port 8787:
   `ssh -i "my-key.pem" ubuntu@35.95.2.5 -L 8787:localhost:8787`

3.2 Remaining 4 Instances Setup

We will run our "Dask workers" on the remaining 4 instances. We need a way for the workers to communicate with the scheduler and this is achieved by passing in the IP address of the scheduler to each worker. This is done in terminal 2 below. We can find the address of the "Dask scheduler" from the output in terminal 3 of our first EC2 instance. For example, the scheduler address is `tcp://172.31.2.184:8786`.

```
[ubuntu@ip-172-31-2-184:~$ source dask_env/bin/activate                    ]
[(dask_env) ubuntu@ip-172-31-2-184:~$ dask scheduler --host 0.0.0.0        ]
2023-04-21 22:46:14,656 - distributed.scheduler - INFO - ------------------
------------------------
2023-04-21 22:46:26,586 - distributed.http.proxy - INFO - To route to workers di
agnostics web server please install jupyter-server-proxy: python -m pip install
jupyter-server-proxy
2023-04-21 22:46:26,630 - distributed.scheduler - INFO - State start
2023-04-21 22:46:26,633 - distributed.scheduler - INFO - ------------------
------------------------
2023-04-21 22:46:26,633 - distributed.scheduler - INFO -   Scheduler at:   tcp:/
/172.31.2.184:8786
2023-04-21 22:46:26,634 - distributed.scheduler - INFO -   dashboard at:  http:/
/172.31.2.184:8787/status
```

Figure 7: Dask Scheduler Address

Repeat the following for each of your 4 EC2 instances. Only the IP address to SSH will change.

a. **Terminal 1 - Dataset transfer**

(a) Open a terminal and SSH into the instance:
`ssh -i "my-key.pem" ubuntu@35.95.2.5`

(b) Copy your AWS API Keys (from the link on ETS dashboard) and paste them in the terminal.

(c) Start dataset transfer:
`aws s3 sync s3://dsc102-pa1-public .`

b. **Terminal 2 - Start Dask Worker**

(a) Open a terminal on your machine and SSH into the instance
`ssh -i "my-key.pem" ubuntu@<worker-ip>`

(b) Activate the dask environment
`source dask_env/bin/activate`

(c) Start 4 Dask worker processes on this machine. Replace scheduler address with yours.
`dask worker tcp://172.31.2.184:8786 --nworkers 4`

3.3 Sanity Check

After completing the above steps, when you visit the "Dask dashboard" at `http://localhost:8787`, you should see 16 workers in total.

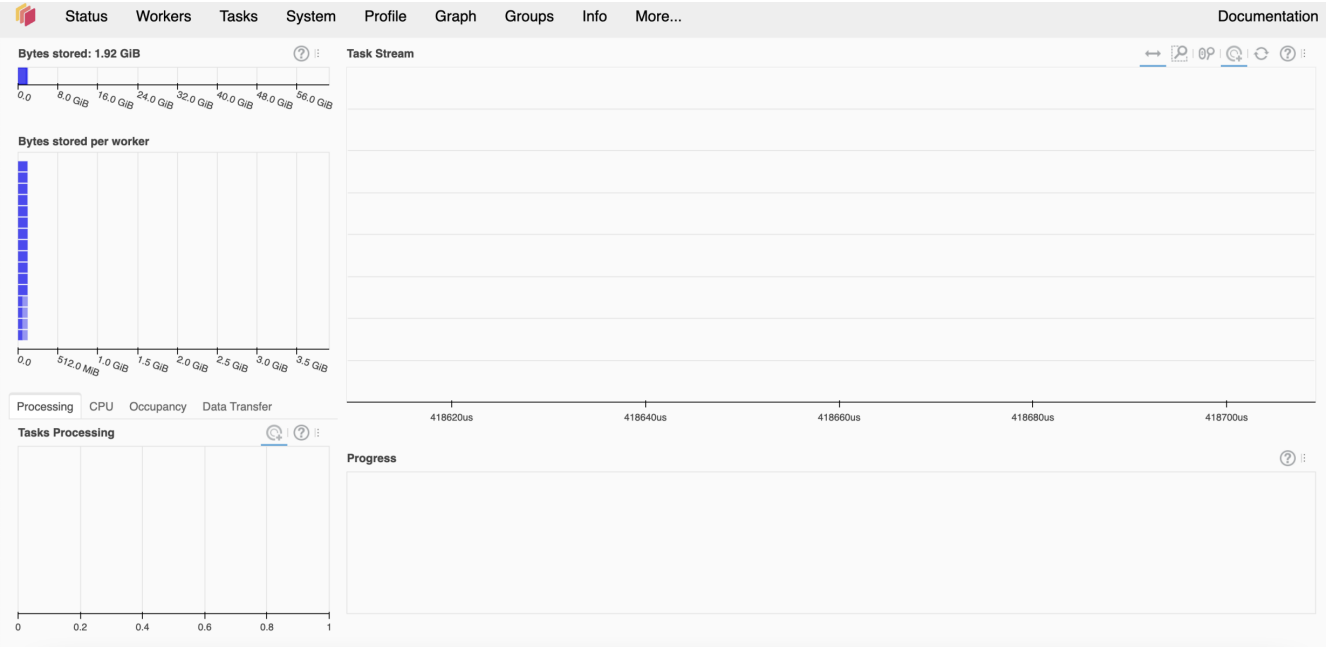


Figure 8: Dask Dashboard with 16 Workers

Scheduler tcp://172.31.2.184:8786

Logs | Exceptions | Bokeh

Workers

Worker	Name	Cores	Memory	Memory use	Occupancy	Processing	In-memory	Services	Logs	Last seen
tcp://172.31.12.172:37127	tcp://172.31.12.172:37127	1	3.91 GiB		0.00 us	0	0	dashboard	logs	909.91 ms
tcp://172.31.12.172:37895	tcp://172.31.12.172:37895	1	3.91 GiB		0.00 us	0	0	dashboard	logs	906.63 ms
tcp://172.31.12.172:42221	tcp://172.31.12.172:42221	1	3.91 GiB		0.00 us	0	0	dashboard	logs	906.25 ms
tcp://172.31.12.172:43499	tcp://172.31.12.172:43499	1	3.91 GiB		0.00 us	0	0	dashboard	logs	907.28 ms
tcp://172.31.14.185:33763	tcp://172.31.14.185:33763	1	3.91 GiB		0.00 us	0	0	dashboard	logs	197.28 ms
tcp://172.31.14.185:41667	tcp://172.31.14.185:41667	1	3.91 GiB		0.00 us	0	0	dashboard	logs	195.39 ms
tcp://172.31.14.185:43197	tcp://172.31.14.185:43197	1	3.91 GiB		0.00 us	0	0	dashboard	logs	196.13 ms
tcp://172.31.14.185:43637	tcp://172.31.14.185:43637	1	3.91 GiB		0.00 us	0	0	dashboard	logs	195.13 ms
tcp://172.31.4.5:34285	tcp://172.31.4.5:34285	1	3.91 GiB		0.00 us	0	0	dashboard	logs	999.63 ms
tcp://172.31.4.5:35269	tcp://172.31.4.5:35269	1	3.91 GiB		0.00 us	0	0	dashboard	logs	1.00 s
tcp://172.31.4.5:39791	tcp://172.31.4.5:39791	1	3.91 GiB		0.00 us	0	0	dashboard	logs	1.53 ms
tcp://172.31.4.5:44969	tcp://172.31.4.5:44969	1	3.91 GiB		0.00 us	0	0	dashboard	logs	998.47 ms
tcp://172.31.8.59:32967	tcp://172.31.8.59:32967	1	3.91 GiB		0.00 us	0	0	dashboard	logs	110.15 ms
tcp://172.31.8.59:35657	tcp://172.31.8.59:35657	1	3.91 GiB		0.00 us	0	0	dashboard	logs	113.24 ms
tcp://172.31.8.59:40651	tcp://172.31.8.59:40651	1	3.91 GiB		0.00 us	0	0	dashboard	logs	111.31 ms
tcp://172.31.8.59:45317	tcp://172.31.8.59:45317	1	3.91 GiB		0.00 us	0	0	dashboard	logs	109.86 ms

Figure 9: Dask Worker Dashboard

4 Large Language Model Usage

Academic Integrity and LLM Usage Policy

If you decide to use LLMs to assist you in this assignment, read this carefully. You will be required to save all LLM chat logs and submit them along with your assignment; these logs will only be inspected in case an academic integrity violation is suspected.

Official Academic Integrity Policy

Academic integrity:

- It is okay to discuss the assignment with your peers at a conceptual level and do team coding within a team. You may use AI models to generate, check, or improve code, provided you attach your complete AI conversation history to your assignment submission. You are responsible for understanding all submitted code and the principles of scalable systems. You may also post conceptual or high-level questions, logistical questions, and useful references on Piazza. However:
  - Do not share any code across teams.
  - Do not post any of your solution code for discussion. Each team is responsible for its submitted code, including any AI-generated code.
- Please review UCSD's honor code and policies and procedures on academic integrity.
- Do not search for code posted online by other students or prior editions. Code analysis tools will be used to compare your submissions, and these go well beyond basic string or syntactic comparisons to catch plagiarism.
- If plagiarism is detected in your code, or if any other form of academic integrity violation is identified, you will receive zero for that component and your overall grade may be downgraded substantially. University authorities will also be notified for appropriate disciplinary action, up to and including expulsion from the University.